



华南理工大学
South China University of Technology

研究生课程论文

Review Report: LLMorpheus: Mutation Testing Using Large Language Models

研究生：徐宇鸿

提交日期：2026 年 05 月 06 日

研究生签名：

学 号	202267330269	学 院	计算机科学与工程学院
课程编号		课程名称	软件测试与质量控制
学位类别	☐ 博士 ☐ 硕士	任课教师	李方
学习形式	☐ 全日制 ☐ 非全日制	学 期	2025-2026 学年第二学期

教师评语：

成绩评定：

分

任课教师签名：

年

月

日

LLMorpheus: 基于大语言模型的变异测试

课程报告

Yuhong Xu

1. 摘要

变异测试通过向程序注入人工故障并观察测试套件能否发现这些故障，来衡量测试质量。传统变异测试工具通常依赖固定的变异算子，例如替换算术运算符、修改条件表达式或删除语句。这类方法实现简单、结果稳定，但也存在明显局限：真实软件缺陷往往涉及错误的方法调用、错误的属性访问、事件名称误用、参数遗漏等语义层面的变化，而固定算子很难覆盖这些缺陷模式。

本文评述 Frank Tip、Jonathan Bell 和 Max Schaefer 发表在 *IEEE Transactions on Software Engineering* 2025 年第 51 卷第 6 期的论文“LLMorpheus: Mutation Testing Using Large Language Models”。该论文提出了一种基于大语言模型的变异测试方法：先用规则在 JavaScript/TypeScript 源代码的指定 AST 位置插入 PLACEHOLDER，再通过提示工程让大语言模型生成可能引入缺陷的替换代码，最后借助修改版 StrykerJS 执行变异体并分类。实验表明，LLMorpheus 能够生成传统工具难以产生的语义型变异体，在 40 个真实缺陷案例中有 10 个完全复现了缺陷代码，另有 26 个产生了与真实缺陷相同的测试失败。该工作说明，大语言模型不仅可以用于代码生成，也可以作为一种上下文敏感的“通用变异算子”，为软件测试质量评估提供新的技术路线。

2. 引言

软件测试的难点之一在于判断一组测试是否真的充分。语句覆盖率、分支覆盖率等指标只能说明代码被执行过，却不能说明测试是否能够发现错误。变异测试从另一个角度衡量测试质量：如果测试套件能够杀死大量被注入故障的程序版本，就说明它对潜在缺陷较为敏感；如果大量变异体存活，则说明测试可能存在断言不足、边界覆盖不足或场景缺失等问题。

传统变异测试建立在“称职程序员假设”和“耦合效应”之上。前者认为真实有缺陷程序通常与正确程序非常接近，后者认为复杂缺陷往往可以由若干简单缺陷组合而成。因此，传统工具使用一组有限的简单变异算子近似真实缺陷，例如将 + 替换为 -，将条件

取反，或删除函数体。这一路线在实践中已被 Pitest、Major、Stryker 等工具采用，也在工业界得到应用。

然而，随着现代软件系统复杂度提高，仅靠固定算子会遗漏许多真实缺陷。论文给出的例子很有代表性：zip-a-folder 的权限检查缺陷涉及属性访问表达式变化，countries-and-timezones 的负数处理缺陷涉及函数调用插入，image-downloader 的路径处理缺陷涉及方法调用替换，事件驱动程序中的缺陷还可能来自事件名错误绑定。这些变更都不是简单的运算符替换，若为每一种缺陷模式都设计专门算子，工具复杂度和执行成本都会迅速上升。

LLMorpheus 的研究动机正来自这一矛盾：变异测试需要更丰富、更接近真实缺陷的变异体，但传统算子集合又不能无限扩张。该论文的关键思路是把“在哪变异”和“怎样变异”分开处理。变异位置仍由规则控制，从而保持可解释性和工程可控性；变异内容则交给大语言模型，根据代码上下文生成语义合理的替换片段。这使变异测试从固定规则驱动转向规则定位与模型生成相结合。

3. 背景

3.1. 变异测试的基本思想

给定程序 P 和测试套件 T ，变异测试工具会对 P 施加小规模修改，生成一组变异体 $M = \{m_1, m_2, \dots, m_n\}$ 。如果某个变异体运行测试后出现测试失败，则该变异体被称为 killed；如果测试全部通过，则称为 survived；如果执行超时，则称为 timed-out。常见的变异得分可以表示为：

$$MS = \frac{K}{K + S},$$

其中 K 表示被杀死的变异体数量， S 表示存活变异体数量。分数越高，通常意味着测试套件越能区分原程序和带缺陷程序。

存活变异体并不总是代表测试不足，因为有些变异体与原程序语义等价，任何测试都无法区分它们。这类等价变异体是变异测试长期面临的问题。一个理想的变异测试工具应当在生成足够多有意义变异体的同时，尽量减少等价变异体和无效变异体。

3.2. 传统变异算子的局限

传统工具的优势在于确定性强、实现清晰、执行流程成熟，但它们通常只覆盖局部语法层面的变化。论文指出，真实缺陷并不总能映射到这些算子。例如，开发者可能调用了相似但语义不同的方法，可能漏传一个关键参数，可能把事件监听器绑定到错误事

件，也可能在属性访问上使用了错误常量。这些错误与上下文关系紧密，只有理解代码意图后才容易构造。

如果继续沿着固定算子路线扩展，需要不断为每类语言结构和 API 模式增加新算子。这不仅增加实现负担，也会产生大量低价值变异体，拖慢测试执行。更重要的是，固定算子只能枚举研究者预先想到的错误形式，而真实缺陷空间远大于算子集合。

3.3. 大语言模型带来的机会

大语言模型在大量自然语言和代码语料上训练，具备一定的代码补全、API 使用和上下文建模能力。LLMorpheus 并没有把大语言模型直接用于生成测试，而是把它用作变异内容生成器。与基于真实缺陷训练专用模型的方法相比，这种方式不需要为每个项目收集缺陷数据和重新训练模型；与传统变异算子相比，它又可以生成更大粒度、更语义化的代码替换。

这种思路的关键风险也很明显。模型可能生成语法错误代码，可能生成与原代码等价的重写，也可能生成不稳定、不可复现的结果。因此，论文没有只提出概念，而是围绕有效变异体数量、等价变异体比例、温度参数、提示模板、模型选择、运行成本和真实缺陷模拟能力设计了系统实验。

4. 方法

4.1. 总体框架

LLMorpheus 可以概括为一个三阶段流程：首先，提示生成器解析源代码，在预定义位置插入 PLACEHOLDER；其次，大语言模型根据带占位符的上下文和原始代码片段生成多个候选替换；最后，变异体生成器过滤无效结果，并交给修改版 StrykerJS 执行测试。该流程既保留传统变异测试“逐个变异并运行测试”的基本框架，也引入了 LLM 的上下文生成能力。

4.2. 变异位置选择

LLMorpheus 使用 BabelJS 解析 JavaScript/TypeScript 源文件，并在若干类型的 AST 节点处插入 PLACEHOLDER。这些位置包括 if、switch、while、do-while 的条件表达式，循环初始化和更新器，函数调用的接收者、参数以及完整参数序列等。选择这些位置的原因是它们与真实缺陷高度相关，且局部替换后仍容易保持语法结构完整。

这一步体现了论文方法中的保守性。LLM 并不是任意改写整个文件，而是在工具指定的位置补全缺陷片段。规则负责控制搜索空间，模型负责在局部上下文内提出更丰

表 1 LLMorpheus 的主要模块

模块	作用
Prompt Generator	解析 JavaScript/TypeScript 源文件，基于 AST 选择变异位置，生成包含 PLACEHOLDER 的提示。
Mutant Generator	从 LLM 返回结果中提取代码块，去除重复、原样返回和语法错误的候选，生成可执行的变异体列表。
Modified StrykerJS	读取预计算变异体，运行项目测试套件，将变异体分类为 killed、survived 或 timed-out，并生成报告。

富的替换方式。这样既降低了生成成本，也减少了完全无关或破坏程序结构的输出。

4.3. 提示设计

默认提示模板包含四类信息：变异测试背景说明、带 PLACEHOLDER 的源代码片段、被替换的原始代码片段，以及要求模型生成有缺陷替换代码的指令。模型被要求为每个占位符给出最多 3 个候选，并用代码块包裹输出，便于后处理程序提取。

提示设计是论文的重要实验变量。作者比较了完整提示、简化系统提示、去除解释、去除详细指令、只要求一个变异和基础填空式提示等多种变体。结果显示，完整提示整体表现最好，而只要求一个变异或只做基础填空会显著减少有效变异体数量。这说明在软件工程任务中，LLM 的能力不仅来自模型本身，也强烈依赖任务描述、上下文组织和输出约束。

4.4. 变异体过滤与执行

LLM 的输出不能直接用于变异测试。LLMorpheus 首先用正则表达式提取 fenced code block 中的候选代码，然后过滤与原始片段完全相同的结果，去除重复候选，再用 BabelJS 检查语法合法性。通过这些检查的候选会被写入 `mutants.json`，供修改版 StrykerJS 使用。

执行阶段沿用 StrykerJS 的测试运行与报告机制。每个变异体被单独应用到源代码并运行测试套件，根据结果划分为 killed、survived 或 timed-out。这样的设计降低了工具实现风险：LLMorpheus 不需要重新实现完整的变异测试执行框架，而是把创新集中在变异体生成环节。

5. 实现

5.1. 工具实现

LLMorpheus 面向 JavaScript/TypeScript 生态实现，主要依赖 BabelJS、Handlebars 和 StrykerJS。BabelJS 用于构建和修改 AST，Handlebars 用于实例化提示模板，StrykerJS 则承担变异体执行、测试运行和报告生成。论文还考虑了实际工程使用中的问题，例如通过 `--rateLimit` 控制请求频率，通过 `--nrAttempts` 处理 API 限流重试，并限制提示中的源码上下文长度以控制 token 成本。

从工程角度看，该实现的价值在于“最小替换”。它没有推翻成熟的变异测试工具链，而是在现有 StrykerJS 流程前增加 LLM 生成变异体的前置阶段。这种架构使方法更容易验证，也便于与传统 StrykerJS 变异算子进行对比。

5.2. 实验设置

论文在 13 个 JavaScript/TypeScript 项目上评估 LLMorpheus，并使用 5 个大语言模型：codellama-34b-instruct、codellama-13b-instruct、llama-3.3-70b-instruct、mixtral-8x7b-instruct 和 gpt-4o-mini。实验围绕 7 个研究问题展开，覆盖变异体数量、等价变异体比例、温度影响、提示模板影响、模型差异、运行成本和真实 bug 模拟能力。

主实验使用 codellama-34b-instruct，温度设为 0.0。LLMorpheus 产生 9,967 个提示，最终得到 6,712 个有效变异体，其中 3,237 个被测试杀死，3,155 个存活，320 个超时。与 StrykerJS 相比，LLMorpheus 生成了更多存活变异体，说明它能够暴露传统算子不容易触及的测试薄弱点。

表 2 主实验中 LLMorpheus 的变异体结果

指标	数量	比例
有效变异体	6,712	100%
killed	3,237	48.2%
survived	3,155	47.0%
timed-out	320	4.8%

5.3. 结果分析

论文对存活变异体进行了人工检查。LLMorpheus 的存活变异体中约 80% 是非等价变异体，20% 是等价变异体；作为对比，StrykerJS 的等价变异体比例约为 5%。这说明 LLM 生成的变异体语义更丰富，但也更容易产生等价重写，例如 `substring`、`substr` 和 `slice` 的近等价替换，或 JavaScript 中会被忽略的多余参数。

温度实验显示，温度为 0.0 时结果最稳定；当温度升高到 0.25、0.50 和 1.0 时，总体变异体数量变化不大，但重复实验之间的一致性明显下降，尤其是温度 1.0 时几乎每次运行都会产生不同结果。模型对比实验显示，`llama-3.3-70b-instruct` 和 `codellama-34b-instruct` 通常生成最多有效变异体和存活变异体，但其他模型也能产生有用结果。

真实缺陷案例研究是论文最有说服力的部分。作者选取 40 个真实 bug，在修复版本附近生成变异体，并比较变异体与原始 buggy 版本的测试失败情况。结果显示，LLMorpheus 在 10 个案例中生成了与真实缺陷语法完全相同的代码，在另外 26 个案例中生成了导致相同测试失败的变异体。这意味着 90% 的真实缺陷至少可以被 LLMorpheus 以行为等价的方式模拟。对于测试质量评估而言，这比“生成很多语法变化”更重要，因为它说明生成的变异体确实接近开发者会犯的错误。

5.4. 局限性

LLMorpheus 仍有几个局限。第一，等价变异体比例高于传统工具，会增加人工审查成本。第二，当前实现主要支持 JavaScript/TypeScript，跨语言推广还需要适配不同语言的解析器和测试框架。第三，占位符策略是预定义的，可能遗漏一些重要变异位置。第四，LLM 输出具有非确定性，即使温度为 0.0，不同模型或服务端实现也可能带来结果差异。第五，真实缺陷复现实验虽然通过多模型比较缓解了训练集泄漏疑虑，但仍难以完全排除模型见过相关代码的可能性。

6. 结论

LLMorpheus 的核心贡献不是简单地把大语言模型接入变异测试，而是提出了一种清晰的分工方式：规则决定变异位置，LLM 生成变异内容，传统变异测试框架负责执行和评估。这个设计既利用了大语言模型对代码上下文和常见缺陷模式的建模能力，也保留了变异测试流程的可控性。

实验结果表明，LLMorpheus 可以生成大量有效变异体，能够模拟许多传统 StrykerJS

算子无法覆盖的真实 bug，并且整体运行成本较低。特别是在 40 个真实缺陷案例中，36 个案例被完全或行为层面复现，说明该方法生成的变异体不只是随机代码扰动，而是具有实际测试价值的缺陷近似。

与此同时，论文也提醒我们不能把 LLM 生成结果直接视为可靠答案。等价变异体、语法无效候选、非确定性和语言生态限制都需要工程过滤和实验验证。LLMorpheus 的价值在于提供了一条务实路线：让 LLM 扩展传统工具的语义覆盖范围，同时用静态解析、测试执行和对比实验约束模型的不确定性。

7. 启发

这篇论文给我的第一点启发是，LLM 在软件工程中的有效用法不一定是端到端替代传统工具。LLMorpheus 没有让模型独立判断测试质量，而是把模型放在最适合发挥作用的位置：根据上下文生成候选缺陷。位置选择、语法校验和测试执行仍由确定性工具完成。这种“传统框架 + LLM 局部增强”的模式比完全依赖模型更容易落地。

第二点启发是，研究创新可以来自对旧流程中瓶颈环节的替换。传统变异测试的瓶颈在于固定算子表达能力不足，而不是整个变异测试思想过时。LLMorpheus 没有否定变异测试，而是把固定算子替换为上下文敏感的生成机制。这种改造思路也可以迁移到其他软件工程任务中，例如静态分析告警修复、测试断言生成、代码审查建议和缺陷定位。

第三点启发是，提示工程需要被当作实验对象而不是经验技巧。论文通过不同提示模板的消融实验说明，输出格式、上下文信息和候选数量都会显著影响结果。这提示后续使用 LLM 做软件工程研究时，应系统报告 prompt 设计、模型配置、温度参数和重复实验，而不能只给出单次运行结果。

最后，这项工作也留下了值得继续研究的问题。后续可以探索自动过滤等价变异体的方法，例如结合静态分析、符号执行或 LLM 自解释；也可以研究更灵活的 PLACEHOLDER 策略，让用户按项目特点指定变异位置；还可以把 LLMorpheus 推广到 Python、Java 等语言，并与自动测试生成工具结合，形成“生成测试-生成变异体-反馈改进测试”的闭环。总体来看，LLMorpheus 说明大语言模型可以成为软件测试基础设施的一部分，但前提是把它纳入严格、可验证、可复现的工程流程中。